and TOPIC. The response payload will be a JSON array of the following:

```
{
    qid: int,
    description: string,
    optionOne: string,
    optionTwo: string,
    optionThree: string,
    optionFour: string
}
```

3. *POST /answers* submits the answers and gets the score. The request payload will be a JSON array of the following:

```
{
    qid: int,
    option: int
}
```

And the response payload will be a JSON with the following structure:

```
{
    total: int,
    rights: int
}
```

Domain model and design

The domain model consists of the Question entity. The field *qid* represents the identity and it is system generated. The field's subject, topic and answer will not have any getters and setters as they are hidden from the users of the system.

The repository will be an interface extending the JpaRespository interface, as the Question objects are stored in an RDBMS system and accessed using Java Persistence API. The repository is extended with three custom methods in line with JPQL.

The controller is a REST controller with appropriate HTTP mappings.

Implementation

Step 1: Create a Spring project with Maven support

Choose Spring version 2.7.3, which is the latest version that supports Java 8+. Add *spring-boot-starter-web* for REST support, and *spring-boot-starter-data-jpa* for JPA-based database connectivity, apart from other dependencies for JSON bindings, database drivers, etc.

Given below is the extract of the critical dependencies in the *pom.xml*. Observe that we are using an H2 database in this project, which can be replaced with any other RDBMS.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-core</artifactId>
</dependency>
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
</dependency>
<dependency>
```

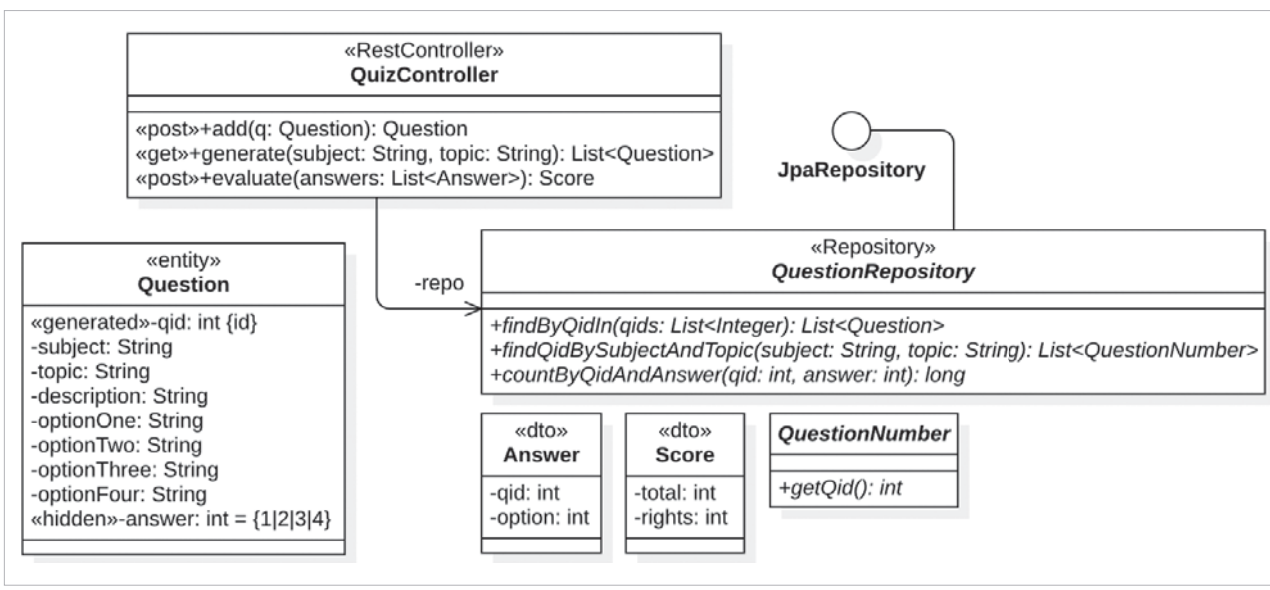

Figure 2: Class model